Algoritmos e Estrutura de Dados II

Aula 07 - Ordenação Interna - MergeSort e QuickSort

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática

Agenda

1. Ordenação por Mistura - MergeSort
2. Ordenação Rápida - QuickSort

Ordenação por Mistura - *Merge Sort*

- O *merge sort*, ou ordenação por mistura, é um exemplo de algoritmo de ordenação do tipo dividir-para-conquistar.
- Sua ideia básica consiste em Dividir (o problema em vários sub-problemas e resolver esses sub-problemas através da recursividade) e Conquistar (após todos os sub-problemas terem sido resolvidos ocorre a conquista que é a união das resoluções dos sub-problemas)
- Como o algoritmo do *Merge Sort* usa a recursividade em alguns problemas esta técnica não é muito eficiente devido ao alto consumo de memória e tempo de execução.

Ordenação por Mistura - *Merge Sort*

- Os três passos úteis dos algoritmos dividir-para-conquistar que se aplicam ao *merge sort* são:
  1. **Dividir**: Dividir os dados em subsequências pequenas;
  2. **Conquistar**: Classificar as duas metades recursivamente aplicando o **merge sort**;
  3. **Combinar**: Juntar as duas metades em um único conjunto já classificado.

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]
[	[1	3]	[5]	[2	4]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]
[	[1	3]	[5]	[2	4]	]
[	[1	3	5]	[2	4]	]

Ordenação por Mistura - *Merge Sort*

	1	2	3	4	5	
[	3	1	5	4	2	]
[	[3	1	5]	[4	2]	]
[	[3	1]	[5]	[4	2]	]
[	[3]	[1]	[5]	[4]	[2]	]
[	[1	3]	[5]	[2	4]	]
[	[1	3	5]	[2	4]	]
[	**1**	**2**	**3**	**4**	**5**	]

Ordenação por Mistura - *Merge Sort*

```
1  void MergeSort(TipoItem *A,
2                 TipoIndice inicio,
3                 TipoIndice fim){
4
5   if (inicio < fim) {
6    int meio = (inicio + fim) / 2;
7    MergeSort(A, inicio, meio);
8    MergeSort(A, meio + 1, fim);
9    merge(A, inicio, meio, fim);
10  }
11
12 }
```

Ordenação por Mistura - *Merge Sort*

```
1  void merge(TipoItem A[],TipoIndice inicio, TipoIndice meio,TipoIndice fim){
2
3    int n1,n2,i,j,k;
4    n1=meio-inicio+1;
5    n2=fim-meio;
6
7    TipoItem A1[n1+1],A2[n2+1];
8
9    for(i=0; i<n1; i++)
10     A1[i]=A[inicio+i];
11   for(j=0; j<n2; j++)
12     A2[j]=A[meio+j+1];
13
14   A1[i].Chave=9999;
15   A2[j].Chave=9999;
16
17   i=0;
18   j=0;
19
20   for(k=inicio; k<=fim; k++) {
21     if(A1[i].Chave<=A2[j].Chave)
22       A[k]=A1[i++];
23     else
24       A[k]=A2[j++];
25   }
26 }
```

Ordenação por Mistura - *Merge Sort*

- Número de Comparações
  - Melhor Caso: $O(n \log n)$
  - Pior Caso: $O(n \log n)$
- Número Movimentações
  - Melhor Caso: $O(n \log n)$
  - Pior Caso: $O(n \log n)$

Ordenação por Mistura - *Merge Sort*

- Vantagens:
  - *MergeSort* é $O(n \log n)$
  - Indicado para aplicações que exigem restrição de tempo (executa sempre em um determinado tempo para um dado n)
  - Passível de ser transformado em estável
  - Fácil Implementação
- Desvantagens:
  - Utiliza funções recursivas
  - Gasto extra de memória, o algoritmo cria uma cópia do vetor para cada nível da chamada recursiva, totalizando um uso adicional de memória igual a $O(n)$

Exercício

- Dada a sequência de números: [3 4 9 2 5 1 8] Ordene em ordem crescente utilizando o algoritmo *MegeSort*, apresentado a sequência dos números a cada passo.

Quick Sort

- Proposto por Hoare em 1960 e publicado em 1962.
- É o algoritmo de ordenação interna mais rápido que se conhece para uma ampla variedade de situações.
- Provavelmente é o mais utilizado.
- A ideia básica é dividir o problema de ordenar um conjunto com n itens em dois problemas menores.
- Os problemas menores são ordenados independentemente.
- Os resultados são combinados para produzir a solução final.

Quick Sort

- A parte mais delicada do método é o processo de partição.
- O vetor $A[Esq..Dir]$ é rearranjado por meio da escolha arbitrária de um **pivô** x.
- O vetor A é particionado em duas partes:
  - Parte esquerda: chaves $\leq$ x.
  - Parte direita: chaves $\geq$ x.

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	<u>3</u>	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	<u>3</u>	1	**5**	4	2	]
2	5	[	3	<u>1</u>	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]

Quick Sort

Esq: 1
Dir: 5
Pivô: 5

i	j		1	2	3	4	5	
1	5	[	3	1	**5**	4	2	]
1	5	[	3	1	**5**	4	2	]
2	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
3	5	[	3	1	**5**	4	2	]
4	4	[	3	1	2	4	**5**	]
4	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]
5	4	[	3	1	2	4	**5**	]
5	4	[	**3**	**1**	**2**	**4**	**5**	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]
1	3	[	3	**1**	<u>2</u>	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]
1	3	[	3	**1**	<u>2</u>	4	5	]
1	2	[	3	<u>**1**</u>	2	4	5	]

Quick Sort

Esq: 1
Dir: 4
Pivô: 1

i	j		1	2	3	4	5	
1	4	[	3	**1**	2	4	5	]
1	4	[	<u>3</u>	**1**	2	4	5	]
1	4	[	3	**1**	2	<u>4</u>	5	]
1	3	[	3	**1**	<u>2</u>	4	5	]
1	2	[	3	<u>**1**</u>	2	4	5	]
2	1	[	**1**	**3**	**2**	**4**	**5**	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]
2	4	[	1	3	**2**	<u>4</u>	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]
2	4	[	1	3	**2**	<u>4</u>	5	]
2	3	[	1	3	<u>**2**</u>	4	5	]

Quick Sort

Esq: 2
Dir: 4
Pivô: 2

i	j		1	2	3	4	5	
2	4	[	1	3	**2**	4	5	]
2	4	[	1	<u>3</u>	**2**	4	5	]
2	4	[	1	3	**2**	<u>4</u>	5	]
2	3	[	1	3	<u>**2**</u>	4	5	]
3	2	[	**1**	**2**	**3**	**4**	**5**	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]
3	4	[	1	2	**3**	<u>4</u>	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**3**	4	5	]
3	3	[	1	2	**3**	4	5	]

Quick Sort

Esq: 3
Dir: 4
Pivô: 3

i	j		1	2	3	4	5	
3	4	[	1	2	**3**	4	5	]
3	4	[	1	2	**<u>3</u>**	4	5	]
3	4	[	1	2	**3**	<u>4</u>	5	]
3	3	[	1	2	**<u>3</u>**	4	5	]
4	2	[	**1**	**2**	**3**	**4**	**5**	]

Quick Sort

```
1  void QuickSort(TipoItem *A, TipoIndice esquerda, TipoIndice direita){
2
3   TipoItem temp, pivo;
4   TipoIndice i, j;
5   i = esquerda;
6   j = direita;
7   pivo = A[(esquerda + direita) / 2];
8
9   while(i <= j){
10
11    while(A[i].Chave < pivo.Chave && i < direita) i++;
12    while(A[j].Chave > pivo.Chave && j > esquerda)j--;
13
14    if(i <= j){
15      temp = A[i];
16      A[i] = A[j];
17      A[j] = temp;
18      i++;
19      j--;
20    }
21  }
22
23  if(j > esquerda)
24   QuickSort(A, esquerda, j);
25
26  if(i < direita)
27   QuickSort(A,  i, direita);
28 }
```

Quick Sort

- Número de Comparações
  - Melhor Caso: $O(n \log n)$
  - Pior Caso: $O(n^2)$
- Número Movimentações
  - Melhor Caso: $O(n \log n)$
  - Pior Caso: $O(n \log n)$

Quick Sort

- Vantagens:
  - É extremamente eficiente para ordenar arquivos de dados.
  - Necessita de apenas uma pequena pilha como memória auxiliar.
  - Requer cerca de $n \log n$ comparações em média para ordenar n itens.
- Desvantagens:
  - Tem um pior caso $O(n^2)$ comparações.
  - Sua implementação é muito delicada e difícil: Um pequeno engano pode levar a efeitos inesperados para algumas entradas de dados.
  - O método não é estável.

Exercício

- Dada a sequência de números: [3 4 9 2 5 1 8] Ordene em ordem crescente utilizando o algoritmo *QuickSort*, apresentado a sequência dos números a cada passo.

Dúvidas?